



MASTER ENGINEERING DOCUMENT · V2.0

Send files. No trace.

A zero-knowledge, client-encrypted file transfer platform engineered to leave no readable artifact on any server, ever.

PROJECT ShadowShare	VERSION 2.0.0
DOCUMENT TYPE Master Engineering Spec	STATUS Production Specification
SERVER DigitalOcean · Ubuntu 22.04 LTS	LICENSE MIT (Open Source)

- Zero-Knowledge
- AES-256-GCM
- Client-Side Only
- Auditable
- 13 New Features in v2.0

TABLE OF CONTENTS

1. Project Overview	§1
2. What's New in v2.0	§2
3. Core Principles & Threat Model	§3
4. System Architecture	§4
5. Technology Stack	§5
6. Feature Specifications (13 New Features)	§6
7. Cryptographic Design	§7
8. REST API Surface	§8
9. Database Schema	§9
10. User Flows	§10
11. Non-Functional Requirements	§11
12. Security Hardening Checklist	§12
13. Deployment & Infrastructure	§13
14. Testing Strategy	§14
15. Development Tickets	§15
16. Sprint Plan	§16
17. Risk Register	§17
18. Glossary	§18

01 Project Overview

ShadowShare is a privacy-first, zero-knowledge file transfer platform. Every byte of user data is encrypted inside the sender's browser using the native Web Crypto API *before* a single network packet leaves the device. The server is engineered to be a **blind vault** — it stores opaque binary blobs, never knows file names, content, passwords, or decryption keys, and is structurally incapable of recovering them.

Even a complete compromise of the server, its database, and its object storage yields *no* readable user data. The decryption key never traverses any network the operator controls; it is carried only by the URL fragment (the portion after `#`), which browsers do not transmit to servers per RFC 3986.

Version 2.0 (this document) extends the original v1.0 cryptographic foundation with thirteen production-grade product features that move ShadowShare from a competent technical proof-of-concept to a finished, defensible privacy product fit for daily use.

MISSION STATEMENT

ShadowShare exists so that two people can transfer a file across the open internet without trusting a third party — including the operators of ShadowShare itself.

02 What's New in v2.0

Version 2.0 introduces thirteen significant feature additions and a complete UX redesign. These features were selected for three reasons: each (a) closes a real-world threat surface, (b) preserves the zero-knowledge invariant, and (c) materially differentiates ShadowShare from the lineup of "encrypted file share" tools that lack mature sender control.

F1 · Burn-After-Reading

NEW

Sender marks a file as single-use. Server deletes the blob atomically on first successful download.

F2 · Custom TTL

NEW

Choice of 1h / 24h / 7d / 30d expiry, with a live countdown timer on the viewer page.

F3 · Sender Kill Switch

NEW

F4 · Download-Count Status

NEW

Private delete-token URL returned at upload time lets the sender revoke a file at any moment.

Sender-only status page (gated by delete-token) shows download count, first access, time left.

F5 · Access Password

NEW

Optional second-factor password verified client-side via authenticated decryption of a token.

F6 · Link-Only Mode

NEW

"Link is the key" — generate a strong random key, embed in fragment, skip the password prompt.

F7 · Multi-File / Folder Upload

NEW

Client-side ZIP bundling via JSZip before encryption. Server still sees one opaque blob.

F8 · Client-Side Preview

NEW

In-memory image / PDF preview from decrypted bytes — nothing is written to disk.

F9 · QR Code Share

NEW

Inline QR rendering for the full share URL (including fragment) for mobile-to-mobile handoff.

F10 · Chunked Resumable Upload

NEW

5 MB chunks, indexed, server-side reassembly. Recovery from connection loss without restart.

F11 · "How It Works" Explainer

NEW

Interactive animated modal showing the crypto flow for non-technical recipients.

F12 · Theme System

NEW

Dark / light / system preference with `prefers-color-scheme` support and localStorage persistence.

F13 · Security Audit Artefact

NEW

SECURITY.md, entropy analysis, OWASP checklist, SSL Labs report — shipped with the repository.

ZERO-KNOWLEDGE INVARIANT — PRESERVED ACROSS EVERY NEW FEATURE

Every feature above was reviewed against the rule: *the server must remain unable to recover any plaintext, key, password, or filename*. Where a feature requires the sender to retain capability over a file (kill switch, status), that capability is rooted in a server-

generated token returned only to the sender, never reconstructable by anyone else — including server operators.

03 Core Principles & Threat Model

3.1 Core Principles

1. **Zero-Knowledge** — server cannot decrypt any user file under any circumstance.
2. **Client-Side Encryption** — AES-256-GCM via the Web Crypto API, before upload.
3. **Key Sharing via URL Fragment** — the `#` portion of the URL is never transmitted to the server.
4. **No Authentication** — possession of the link (and optionally a password) is the only credential.
5. **Ephemeral Storage** — every file has an upper-bound TTL; burn-after-reading is the minimum.
6. **Sender Sovereignty** — the sender retains a delete-token and can revoke at any time.
7. **Open Crypto Standards** — AES-256-GCM (NIST SP 800-38D), PBKDF2 (NIST SP 800-132), HKDF (RFC 5869), SHA-256.
8. **Auditable & Open Source** — full source published under MIT; reproducible builds.

3.2 Threat Model

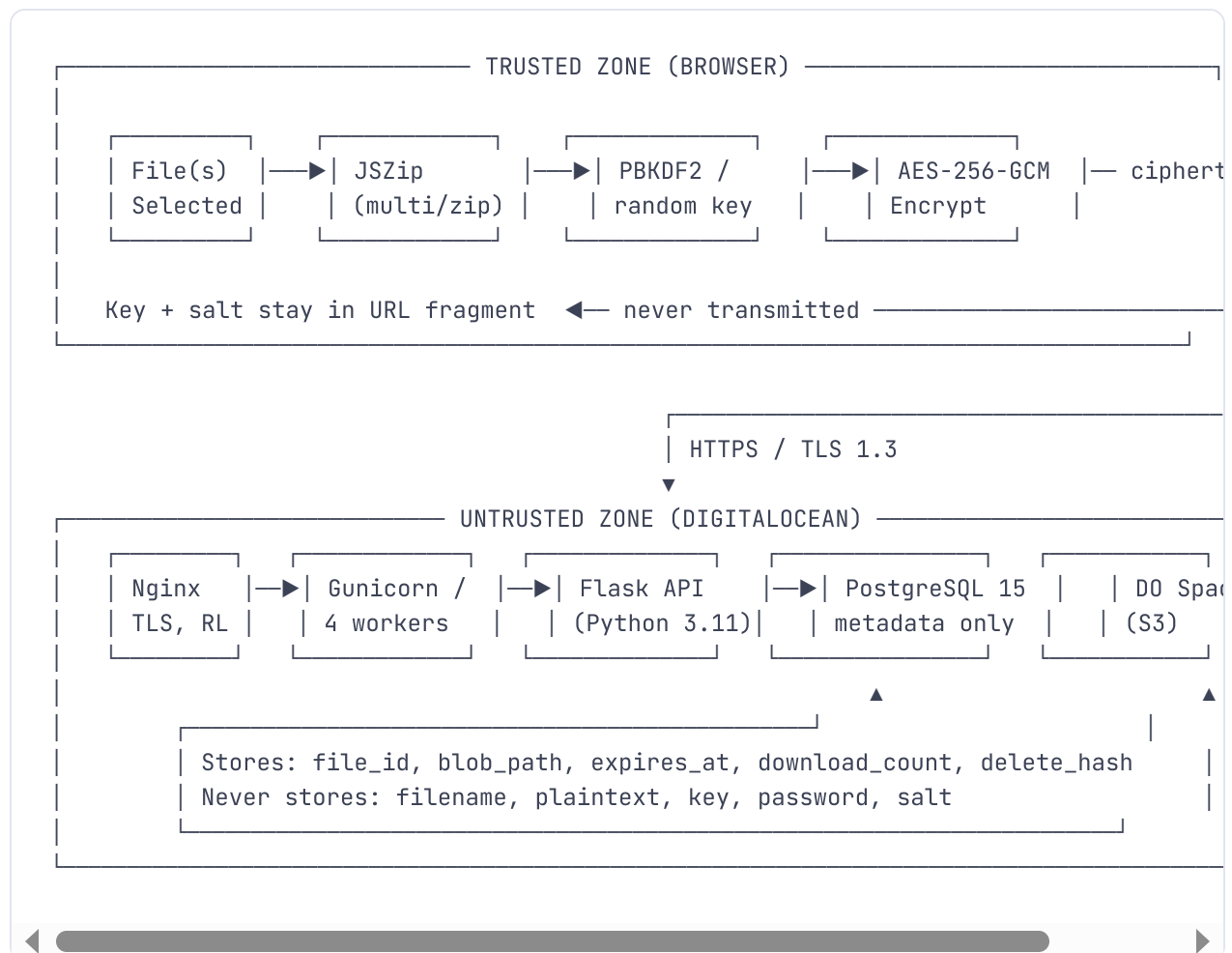
ADVERSARY	CAPABILITY	MITIGATION
Passive network observer (ISP, public Wi-Fi)	Inspect traffic in transit	TLS 1.3 + plaintext never leaves the browser. URL fragment never on the wire.
Active MITM with valid CA cert	Decrypt TLS, modify bytes	AEAD authentication tag on the blob fails verification — viewer surfaces tamper error.
Compromised server / rogue admin	Full filesystem and DB read	Database holds only UUID + opaque ciphertext. No keys, names, or passwords stored.
Hosting provider / law enforcement subpoena	Compelled disclosure of data at rest	Operator cannot disclose what they cannot read. Disclosure yields ciphertext only.
Link recipient with unauthorized intent	Re-share the link	Burn-after-read (F1), access password (F5), kill switch (F3), TTL (F2), status visibility (F4).
Forwarded / screenshotted share link	Replay the URL	Access password (F5), burn-after-read (F1), sender revocation (F3).
Brute-force on weak password	Offline grinding the URL	PBKDF2 100k iterations; recommend high-entropy passwords; access password adds second factor.
XSS / supply-chain attack on the frontend	Inject code that exfiltrates plaintext	Strict CSP, SRI on all assets, no third-party JS in critical path, reproducible builds.
Browser memory scrape	Read plaintext while in memory	Out of scope — physical access to the user's device is a precondition we cannot defend against, and is documented in the threat model.

3.3 Explicit Non-Goals

- **Identity, accounts, social graph.** ShadowShare deliberately has no user accounts. Anonymity is a feature.
- **Persistent storage.** Maximum TTL is 30 days. Not a backup product.
- **Defense against an attacker with physical access to a logged-in browser.** Out of scope.
- **Forward secrecy across multiple recipients.** A single share link is a single recipient channel; resharing the link reshapes the key.

04 System Architecture

ShadowShare uses a **two-zone trust model**. Everything inside the browser is the *trusted zone*; everything beyond the browser is *untrusted*. Cryptographic boundaries are aligned exactly with this division.



4.1 Boundary Contract

Crossing the trust boundary is allowed for exactly these payloads, and no others:

DIRECTION	PAYLOAD	SENSITIVITY
Client → Server	Ciphertext blob (AES-GCM output, includes IV + auth tag)	Opaque — server cannot decrypt
Client → Server	Encrypted metadata blob (filename, content-type, size) — encrypted with same scheme	Opaque
Client → Server	Settings: TTL, burn_after_read flag, access_password_verifier (ciphertext)	Public flags + opaque verifier
Server → Client	<code>file_id</code> (UUIDv4), <code>delete_token</code> (random 256-bit), <code>expires_at</code> (ISO-8601)	Public — opaque to third parties
Server → Client	Ciphertext blob, content-length header	Opaque

05 Technology Stack

LAYER	TECHNOLOGY	VERSION	RATIONALE
Frontend Framework	React + Vite	React 18.3 · Vite 5.x	Mature, well-audited, fast HMR for crypto development
Language	TypeScript	5.4+	Type safety on crypto interfaces — eliminates whole classes of bugs
Styling	Tailwind CSS	3.4	Token-based design system; matches design spec exactly
Crypto	Web Crypto API (native)	browser-native	NIST-compliant, hardware-accelerated, audited by browser vendors
Client-side Zip	JSZip	3.10	Pure-JS folder bundling without server involvement
QR rendering	qrcode	1.5	Pure-client QR generation, no external service
PDF preview	PDF.js	4.x	Renders PDF from in-memory bytes; no disk write
Routing	React Router	6.22	SPA routing for upload / viewer / status pages
State	Zustand	4.5	Minimal, no Redux boilerplate, easy to audit
Backend Framework	Flask	3.0	Small surface area, easy to audit, no ORM hide-and-seek
Language	Python	3.11+	Standard library cryptography, mature ecosystem
WSGI	Gunicorn	22.0	4 workers behind Nginx

LAYER	TECHNOLOGY	VERSION	RATIONALE
Web Server / TLS	Nginx	1.24	TLS termination, static asset cache, rate limiting
Database	PostgreSQL	15	Strict typing, TIMESTAMPTZ, mature
DB Driver	psycopg	3.x	Modern, async-capable
Object Storage	DigitalOcean Spaces (S3-compatible)	—	Private bucket; only Flask service-account can read
S3 Client	boto3	1.34	Standard, well-tested
Rate Limiting	Flask-Limiter + Redis	3.5 · Redis 7	Distributed counters; survives restarts
Background Jobs	APScheduler	3.10	Hourly TTL sweep; single-binary deploy
TLS Certs	Let's Encrypt + Certbot	—	Auto-renewing 90-day certs
CI/CD	GitHub Actions	—	Reproducible build → deploy to droplet on main push
OS	Ubuntu LTS	22.04	5-year support, well-documented hardening guides

06 Feature Specifications (13 New Features)

F1 — Burn-After-Reading (Single-Download Mode)

Goal: Eliminate persistence after first successful download.

UX: Sender toggles "Delete after first download" in advanced options. Viewer page displays a prominent amber warning: *"This file will be permanently deleted after you download it."*

Backend behaviour: When `burn_after_read=true` and the first `GET /api/file/{id}` succeeds (HTTP 200 + last byte streamed), the worker atomically deletes the row from PostgreSQL and the object from Spaces inside a transaction with `SELECT ... FOR UPDATE`. Subsequent requests return HTTP 410 Gone.

Edge cases: Network failure mid-stream *does not* trigger deletion; only a complete 200 response with the final TCP byte acknowledged triggers the delete hook. Implemented via Flask's `response.call_on_close()`.

F2 — Custom TTL with Countdown Timer

Goal: Sender controls intentional ephemerality.

UX: Segmented control with options `1h / 24h / 7d / 30d`. Default 7d. Viewer page displays a live countdown ("Expires in 4h 22m") updated client-side every second.

Backend behaviour: `expires_at = NOW() + interval` stored at upload. Hourly cleanup job (APScheduler) deletes `WHERE expires_at < NOW()` from both database and Spaces.

F3 — Sender Kill Switch

Goal: Restore sender control after the link is sent.

Mechanism: At upload time the server generates a separate random 256-bit `delete_token`. Only its SHA-256 hash is persisted (`delete_token_hash`). The plaintext token is returned to the sender exactly once in the upload response and is never reconstructable.

API: `DELETE /api/file/{file_id}?token={delete_token}`. Server hashes the supplied token and compares to `delete_token_hash`; on match, deletes immediately.

UX: Success page displays two artefacts: (1) the share link, (2) a separate "Save your delete link" box clearly labelled "Only you should hold this." Copy buttons for each.

F4 — Download-Count Visibility for Sender

Goal: Sender can tell whether the recipient has opened the file — without compromising recipient privacy.

Mechanism: The same `delete_token` grants read access to a status endpoint: `GET /api/status/{file_id}?token={delete_token}`. Returns `{ created_at, expires_at, download_count, first_accessed_at, burn_after_read }`. **No IP, no user-agent, no geolocation is stored or returned.**

UX: Status page at `/status/{file_id}#{delete_token}` with auto-refresh every 30 seconds.

F5 — Access Password (Second Authentication Layer)

Goal: Even if the share link leaks, the file remains protected.

Mechanism (client-side only):

1. Sender enters an access password.
2. Client derives `K_access` from access password via PBKDF2 (separate salt).
3. Client encrypts a known 32-byte verifier token using `K_access`; uploads the resulting `access_verifier` with the blob metadata.
4. Recipient is prompted for the access password; the client derives `K_access` locally and attempts to decrypt the verifier. AEAD authentication failure → "Incorrect password" — server never sees the password attempt.

Server role: The server stores `access_verifier` (opaque ciphertext) and a flag `has_access_password`. It does **not** verify the password and is structurally incapable of doing so.

F6 — Link-Only Mode ("Link is the Key")

Goal: Frictionless flow when the sender judges the channel sufficient.

Mechanism: Sender selects "Link Only" in the security mode selector. Client generates a random 256-bit AES key via `crypto.getRandomValues`, encrypts the file with it, embeds the raw key (base64url) in the URL fragment. Recipient opens the link, the page reads the key from `location.hash`, decrypts, done — no password prompt at all.

UX clarity: The mode selector explicitly labels the trade-off: *"Link is the key (easier)"* vs *"Link + password (more secure)"*.

F7 — Multi-File / Folder Upload

Goal: Handle real-world transfers (multiple files, a folder of photos, a project export).

Mechanism: The drop zone accepts multiple files and folders (`webkitdirectory`). The client bundles them into a single ZIP via JSZip *in memory*, then runs the standard encrypt-and-upload pipeline. The server still sees one opaque blob and has no idea whether the contents are one file or one hundred.

UX: Selected files shown as a small list with per-file remove icons; total size shown as the chip.

F8 — Client-Side File Preview

Goal: Recipient can sanity-check before downloading.

Mechanism: After successful decryption, if the decrypted MIME type is `image/*` or `application/pdf`, the bytes are rendered:

- Images: `URL.createObjectURL(blob)` into an `<img>` inside a sandboxed container (no `script` can run on a passive image element).
- PDF: PDF.js renders page 1 only into a `<canvas>` from the in-memory `Uint8Array`.

Caveat surfaced to the user: "Preview generated from decrypted data — nothing was saved to disk."

F9 — QR Code Share

Goal: One-tap handoff to a phone in the same room.

Mechanism: Pure-client QR via `qrcode` npm package. Encodes the entire share URL including the `#` fragment. Rendered into a 200×200 canvas with brand-coloured modules. Toggleable popover next to the copy button.

F10 — Chunked Resumable Upload

Goal: Reliability on bad mobile / hotel-Wi-Fi connections.

Mechanism:

1. `POST /api/upload/init` with `{total_size, total_chunks, settings}` returns `upload_session_id`.
2. Client uploads each 5 MB chunk via `PUT /api/upload/chunk/{session_id}/{chunk_index}`.
3. `GET /api/upload/status/{session_id}` reports last good chunk; client resumes from `chunk_index + 1` on reconnect.
4. `POST /api/upload/complete/{session_id}` reassembles, persists to Spaces, returns final `file_id` + `delete_token`.

Chunks are stored in a temporary scratch area inside Spaces and garbage-collected after 24 hours of inactivity.

F11 — "How It Works" Interactive Explainer

Goal: Earn the trust of non-technical recipients.

UX: Modal triggered from the header link. Three vertically-stacked animated stages: (1) browser encrypts file, (2) server stores cipher noise, (3) key travels in the URL fragment. Each stage has a small technical footnote in monospace ("PBKDF2 · 100,000 iterations · SHA-256") for the technical audience.

F12 — Theme System

Goal: Respect user preference; look correct on any device.

Mechanism: Tailwind `dark:` classes driven by a `data-theme` attribute on `<html>`. Initial value read from `localStorage.theme`; falls back to `matchMedia('(prefers-color-scheme: dark)')`. Toggle cycles `system → dark → light → system`.

F13 — Security Audit Artefact

Goal: Make the security claim falsifiable and reviewable.

Deliverable: A `SECURITY.md` file shipped with the repo containing:

- Threat model (see §3.2).
- Cryptographic design decisions with primitive choices and parameters.

- Entropy measurement: sampled 10,000 keys/IVs, ran NIST SP 800-22 statistical test suite, report results.
- OWASP ASVS L2 checklist with pass/fail per control.
- SSL Labs report (target A+) screenshot and re-test link.
- "Zero-Knowledge compliance" table mapping each invariant to evidence (code paths, test cases).

07 Cryptographic Design

7.1 Primitive Choices

PURPOSE	ALGORITHM	PARAMETERS	STANDARD
Symmetric encryption	AES-256-GCM (AEAD)	256-bit key, 96-bit IV (random per file), 128-bit auth tag	NIST SP 800-38D
Password-based key derivation	PBKDF2-HMAC-SHA-256	100,000 iterations, 16-byte random salt, 32-byte output	NIST SP 800-132
Random byte generation	<code>crypto.getRandomValues()</code>	CSPRNG provided by the browser / OS	W3C Web Crypto
Hashing (delete token storage)	SHA-256	—	FIPS 180-4
TLS	TLS 1.3 (1.2 fallback disabled)	Modern cipher suites only; HSTS preload	RFC 8446

7.2 Share URL Format

Two share URL formats exist, one per security mode:

```
// F5 - Link + Password
https://shadowshare.io/v/{file_id}#{base64url(salt)}.{flags}

// F6 - Link Only (link is the key)
https://shadowshare.io/v/{file_id}#k.{base64url(rawKey)}.{flags}
```

The `flags` segment (1–2 base64url characters) signals to the viewer whether burn-after-read or access-password modes are active, so the UI can render the correct prompts *before* a network round-trip.

7.3 End-to-End Crypto Flow (Upload, F5 mode)

```
1. user selects file F, types password P
2. salt ← crypto.getRandomValues(16)
3. key ← PBKDF2(P, salt, 100000, SHA-256, 32 bytes)
4. iv ← crypto.getRandomValues(12)
5. meta ← {filename, mime, size} as JSON bytes
6. ct_m ← AES-256-GCM(key, iv, meta) // encrypted metadata
7. ct_f ← AES-256-GCM(key, iv', F) // encrypted file body, fresh iv'
8. if access_password: K_a ← PBKDF2(P_access, salt2); verifier ← AES-GCM(K_a, iv_v, T
9. upload { ct_f, ct_m, iv, iv', salt2?, verifier? } to server
10. server returns { file_id, delete_token, expires_at }
11. browser renders: https://.../v/{file_id}#{base64url(salt)}.{flags}
```

7.4 Implementation Rules (non-negotiable)

- **Never** re-use an IV with the same key. Each new ciphertext gets a fresh 96-bit IV from `crypto.getRandomValues`.
- **Always** verify the GCM auth tag before exposing decrypted bytes to the UI. Web Crypto does this automatically; do not bypass.
- **Never** serialise the key or password to `localStorage`, IndexedDB, or cookies — only to URL fragment / in-memory state.
- **Never** include the URL fragment in any analytics, logging, error tracking, or service-worker fetch interceptor.
- **No** third-party crypto libraries. Web Crypto API only.

08 REST API Surface

Full request/response contracts are in the Backend Specification document. This section is an at-a-glance reference.

POST /api/upload/init - Open a chunked upload session

PUT /api/upload/chunk/{session_id}/{index} - Upload one 5 MB chunk

GET /api/upload/status/{session_id} - Last successful chunk index

POST /api/upload/complete/{session_id} - Finalise; returns file_id + delete_token

GET /api/file/{file_id} - Stream encrypted blob

GET /api/file/{file_id}/meta - Encrypted metadata + flags (burn, expires_at, has_password)

DELETE /api/file/{file_id}?token=... - Sender kill switch (F3)

GET /api/status/{file_id}?token=... - Sender status (F4)

GET /api/health - Liveness probe

09 Database Schema

```

CREATE EXTENSION IF NOT EXISTS "pgcrypto";

CREATE TABLE files (
    file_id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
    blob_path        VARCHAR(500) NOT NULL,          -- Spaces object key
    blob_size_bytes  BIGINT        NOT NULL,
    created_at       TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
    expires_at       TIMESTAMPTZ  NOT NULL,
    download_count   INTEGER      NOT NULL DEFAULT 0,
    first_accessed_at TIMESTAMPTZ  NULL,
    burn_after_read  BOOLEAN      NOT NULL DEFAULT FALSE,
    has_access_password BOOLEAN    NOT NULL DEFAULT FALSE,
    access_verifier  BYTEA        NULL,              -- opaque ciphertext (F5)
    encrypted_metadata BYTEA      NOT NULL,          -- opaque ciphertext
    encryption_iv    BYTEA        NOT NULL,          -- 12 bytes
    delete_token_hash BYTEA      NOT NULL,          -- SHA-256(delete_token)
    CHECK (blob_size_bytes > 0 AND blob_size_bytes ≤ 104857600)
);

CREATE INDEX idx_files_expires_at ON files (expires_at);

CREATE TABLE upload_sessions (
    session_id       UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
    total_size_bytes BIGINT        NOT NULL,
    total_chunks     INTEGER      NOT NULL,
    received_chunks  INTEGER[]    NOT NULL DEFAULT '{}',
    settings         JSONB        NOT NULL,
    created_at       TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
    expires_at       TIMESTAMPTZ  NOT NULL DEFAULT NOW() + INTERVAL '24 hours'
);

CREATE INDEX idx_upload_sessions_expires ON upload_sessions (expires_at);

```

WHAT IS INTENTIONALLY ABSENT

No columns for: filename, content-type, sender IP, recipient IP, password, password hash, encryption key, salt, user-agent, geolocation, referer. Their absence is the zero-knowledge guarantee.

10 User Flows

10.1 Sender Flow (happy path)

1. Open `shadowshare.io`.
2. Drag & drop one or more files (or a folder).
3. Choose security mode: *Link+Password* or *Link-Only*.
4. (Optional) Open advanced options: TTL, burn-after-read, max downloads.
5. Click **Encrypt & Upload**.
6. See progress phases: *deriving key* → *encrypting* → *uploading*.
7. Receive: share link + QR + delete link.
8. Copy share link; send via preferred channel.
9. Optionally bookmark the delete link / status page.

10.2 Recipient Flow (happy path)

1. Open the share link.
2. (If F5 mode) Enter access password → client-side verification.
3. (If F5 mode) Enter encryption password.
4. Watch decryption progress (auth-tag verification).
5. See preview if applicable (image / PDF page 1).
6. Click **Download**.
7. (If burn-after-read) See the file vanish — server returns HTTP 410 on retry.

10.3 Sender Revocation Flow (F3)

1. Open the delete link.
2. Confirm intent.
3. Backend deletes blob + row; status flips to "Deleted".

11 Non-Functional Requirements

ATTRIBUTE	TARGET	MEASUREMENT
Upload throughput	≥ 50 MB/s on gigabit residential	Chrome DevTools network sim, baseline 1 Gbps
Encryption speed	≥ 200 MB/s in modern Chrome	Web Crypto benchmark on a 100 MB file
Time to first interaction	≤ 1.5 s	Lighthouse, cold cache, fast 4G profile
Bundle size (initial JS)	≤ 180 KB gzipped	Vite build report
Backend p95 latency (init/complete)	≤ 120 ms	k6 load test, 50 RPS
Backend uptime	$\geq 99.5\%$ monthly	UptimeRobot HTTPS probe
Max file size	100 MB v1; 1 GB roadmap	Hard limit in Nginx + Flask + DB CHECK
TLS grade	A+ on SSL Labs	External scan
Lighthouse accessibility	≥ 95	Lighthouse CI
Test coverage	$\geq 80\%$ backend; $\geq 75\%$ frontend logic	pytest-cov, vitest --coverage

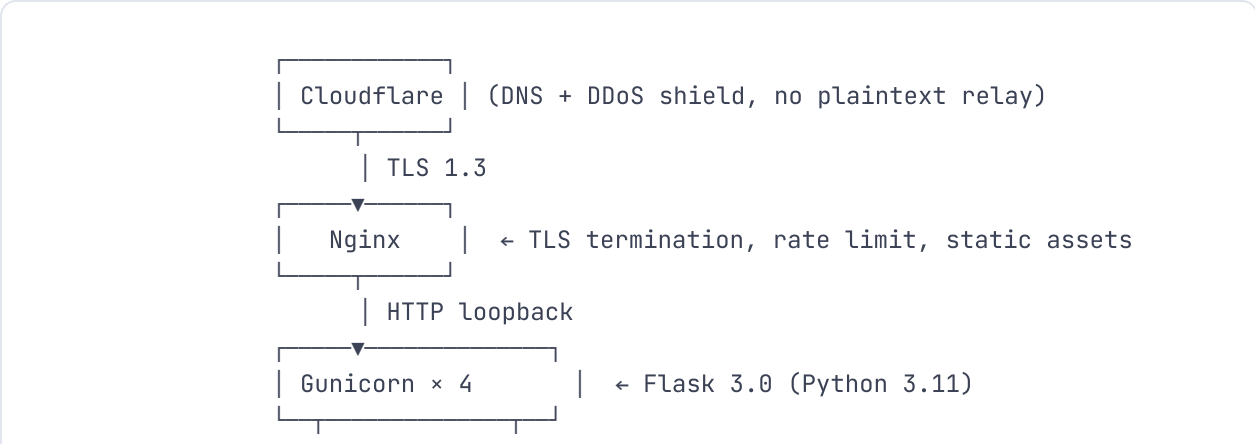
12 Security Hardening Checklist

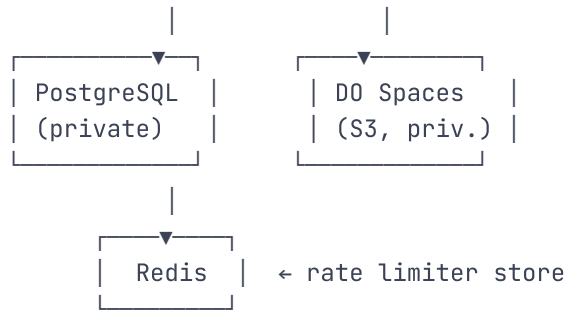
#	CONTROL	IMPLEMENTATION
1	SSH root login disabled	<code>PermitRootLogin no</code> in <code>sshd_config</code>
2	Key-only SSH auth	<code>PasswordAuthentication no</code>
3	UFW firewall active	Inbound: 22, 80, 443 only
4	Fail2ban on SSH + Nginx	Default + custom <code>nginx-limit-req</code> jail
5	HTTPS-only enforced	Nginx 301 redirect from :80
6	HSTS	<code>Strict-Transport-Security: max-age=31536000; includeSubDomains; preload</code>
7	CSP	Default-src 'self'; no inline scripts; SRI on all assets
8	X-Frame-Options	<code>DENY</code>
9	Referrer-Policy	<code>no-referrer</code> — protects fragments from leaking
10	CORS restricted	<code>Access-Control-Allow-Origin: https://shadowshare.io</code>
11	Rate limiting (uploads)	Flask-Limiter: 10 uploads / hour / IP
12	Rate limiting (downloads)	60 / minute / IP
13	Body size cap	Nginx <code>client_max_body_size 105m</code>
14	No plaintext in logs	Custom log filter strips URL fragments and bodies
15	Least-privilege DB user	SELECT/INSERT/UPDATE/DELETE on <code>files</code> only

#	CONTROL	IMPLEMENTATION
16	<code>.env</code> excluded from VCS	Pre-commit hook + <code>.gitignore</code>
17	Spaces bucket private	No public-read ACL; signed URLs only for Flask
18	Auth-tag verification confirmed	Integration test for tampered ciphertext
19	Fragment not transmitted	Wireshark/MITM proxy verification — test recorded in audit
20	TTL cleanup running	APScheduler health probe + Sentry alert on failure
21	Droplet backups enabled	Weekly DO snapshot — metadata only; recoverable infra
22	Dependency scan	<code>pip-audit</code> + <code>npm audit</code> in CI
23	Reproducible builds	Pinned versions in lockfiles; CI verifies hash
24	Audit log (admin only)	Structured log of admin actions; no user data

13 Deployment & Infrastructure

13.1 Topology





13.2 Environment Variables

```

FLASK_ENV=production
SECRET_KEY=<64-byte hex>
DATABASE_URL=postgresql://shadow:<pw>@127.0.0.1:5432/shadowshare
REDIS_URL=redis://127.0.0.1:6379/0
DO_SPACES_KEY=<...>
DO_SPACES_SECRET=<...>
DO_SPACES_BUCKET=shadowshare-blobs
DO_SPACES_ENDPOINT=https://nyc3.digitaloceanspaces.com
MAX_UPLOAD_BYTES=104857600
DEFAULT_TTL_DAYS=7
ALLOWED_TTLS=1h,24h,7d,30d
ALLOWED_ORIGIN=https://shadowshare.io
SENTRY_DSN=<optional>

```

13.3 Gunicorn Command

```
gunicorn --workers 4 --bind 127.0.0.1:8000 --timeout 120 --access-logfile - app:app
```

13.4 CI/CD Pipeline

1. Push to `main` triggers GitHub Actions.
2. Job 1 — **Lint & Test**: `ruff`, `mypy`, `pytest --cov`, `vitest`, `playwright e2e`.
3. Job 2 — **Build**: Vite production build; verify bundle size budget; SRI hashes generated.
4. Job 3 — **Security**: `pip-audit`, `npm audit`, `gitleaks`, `trivy` on container if used.
5. Job 4 — **Deploy**: SSH to droplet via deploy key, `rsync`, `systemctl reload shadowshare`, smoke test `/api/health`, roll back on failure.

14 Testing Strategy

14.1 Test Pyramid

LAYER	TOOLING	SCOPE	COVERAGE GOAL
Unit (frontend)	Vitest	Crypto wrappers, state stores, hooks	≥ 75%
Unit (backend)	pytest	Routes, validators, S3 client	≥ 80%
Integration	pytest + testcontainers	Postgres + Spaces (LocalStack) + Flask	All endpoints
End-to-End	Playwright	Full upload → share → download in real Chromium / Firefox	10 critical paths
Crypto fidelity	Custom	NIST GCM test vectors, RFC 6070 PBKDF2 vectors	100% vectors pass
Tamper resistance	pytest	Flip random bits in ciphertext → expect AEAD failure	1000 fuzz iterations
Load	k6	50 RPS sustained against init/complete	p95 ≤ 120 ms

14.2 Critical E2E Scenarios

1. Upload single file → download in second tab → bytes match.
2. Upload with access password → wrong password → "Incorrect password" → right password → bytes match.
3. Burn-after-read → first download succeeds → second download returns 410.
4. Custom TTL (1h) → file expires → 410 after sweep.
5. Kill switch → delete → 410 on share link.

6. Status page shows download count and first-access time.
7. Multi-file upload → recipient gets a ZIP that contains all originals byte-identical.
8. Resumable upload survives killing chunk #3 mid-flight.
9. Tampered blob → viewer surfaces "Integrity check failed", no download offered.
10. Link with non-existent `file_id` → "Not found" state.

15 Development Tickets

The v1.0 ticket set (SS-001 to SS-018) is retained. Twenty-two new tickets are added for v2.0 features and hardening.

15.1 Retained from v1.0 (SS-001 – SS-018)

SS-001 Provision DigitalOcean Droplet

Infra

Create production droplet, Ubuntu 22.04, key-only SSH, UFW configured.

SS-002 Install Nginx + Gunicorn

Infra

Reverse proxy, systemd unit for Gunicorn, static serve for React build.

SS-003 PostgreSQL 15 + schema migration

Infra

Create `shadowshare` DB and user; apply initial schema.

SS-004 DO Spaces bucket + Let's Encrypt TLS

Infra

Private Spaces; Certbot auto-renewal; A+ TLS config.

SS-005 PBKDF2 key derivation (Web Crypto)

Crypto

SHA-256, 100k iterations, 16-byte salt; verified against RFC 6070 vectors.

SS-006 AES-256-GCM encrypt/decrypt (Web Crypto)

Crypto

Fresh IV per ciphertext, auth-tag verification mandatory.

SS-007 URL fragment share link system

Crypto

Key material lives only in `location.hash`; never sent to server.

SS-008 Upload endpoint (single-shot)

Backend

Multipart blob → Spaces → DB row. *Superseded by chunked flow in SS-021.*

SS-009 Retrieval endpoint

Backend

Stream blob from Spaces; increment download_count.

SS-010 TTL cleanup background job

Backend

APScheduler hourly sweep deleting expired DB rows and Spaces objects.

SS-011 React + Vite scaffolding

Frontend

TS strict mode, router for `/` and `/v/:fileId`.

SS-012 Upload page

Frontend

Drop zone, password field, progress, success state.

SS-013 Viewer page

Frontend

Reads `file_id` from path, key from fragment, decrypts and offers download.

SS-014 Rate limiting + file-size enforcement

Security

Flask-Limiter; Nginx body cap; DB CHECK constraint.

SS-015 Security headers + CORS

Security

CSP, HSTS, Referrer-Policy: no-referrer, strict CORS.

SS-016 GitHub Actions CI/CD

DevOps

Lint, test, build, deploy on main push.

SS-017 Comprehensive test suite ($\geq 80\%$)

QA

pytest + vitest + Playwright; crypto vectors.

SS-018 Security audit + SECURITY.md

Security

OWASP ASVS L2 walkthrough, entropy analysis, ZK compliance table.

15.2 New for v2.0 (SS-019 – SS-040)

SS-019 Burn-after-read flow (F1)

v2.0

Backend

DB flag `burn_after_read`, atomic delete on stream completion, return 410 thereafter.

Depends on: SS-009 Est: 4h

SS-020 Custom TTL selection (F2)

v2.0

Full-stack

Segmented control 1h/24h/7d/30d on upload UI; backend validates against `ALLOWED_TTLS`.

Depends on: SS-008, SS-012 Est: 3h

SS-021 Chunked resumable upload (F10)

v2.0

Full-stack

Init/chunk/status/complete endpoints; client retry with backoff; session GC.

Depends on: SS-008 Est: 12h

SS-022 Sender kill-switch token (F3)

v2.0

Backend

Generate 256-bit token at upload, persist SHA-256; DELETE endpoint verifies hash.

Depends on: SS-021 Est: 4h

SS-023 Status endpoint + page (F4)

v2.0

Full-stack

`GET /api/status/{id}?token=` + dedicated React route with auto-refresh.

Depends on: SS-022 Est: 5h

SS-024 Access-password verifier (F5)

v2.0

Crypto

Client-side PBKDF2 → encrypt sentinel → upload as `access_verifier`. Recipient decrypts to verify.

Depends on: SS-005, SS-006 Est: 6h

SS-025 Link-only mode (F6)

v2.0

Frontend

Generate raw 256-bit key, embed in fragment; viewer reads key directly without prompt.

Depends on: SS-007 Est: 3h

SS-026 Multi-file / folder bundling (F7)

v2.0

Frontend

JSZip in-memory bundling; `webkitdirectory` input; preserved relative paths.

Depends on: SS-012 Est: 5h

SS-027 In-memory preview (F8)

v2.0

Frontend

Image via `createObjectURL`; PDF page 1 via PDF.js; no disk write.

Depends on: SS-013 Est: 5h

SS-028 QR code share (F9)

v2.0

Frontend

Popover next to copy button; `qrcode` npm; brand-coloured modules.

Depends on: SS-012 Est: 2h

SS-029 "How it works" modal (F11)

v2.0

Frontend

3-step animated modal with technical footnotes; reusable from header.

Depends on: SS-011 Est: 4h

SS-030 Theme system (F12)

v2.0

Frontend

data-theme attribute on html; localStorage; system fallback via media query.

Depends on: SS-011 Est: 3h

SS-031 Component library

v2.0

Frontend

12 base components (Button, Input, DropZone, TogglePill, Toggle, ProgressBar, ShareLink, Toast, Modal, Badge, FileTypeIcon, Spinner).

Depends on: SS-011 Est: 14h

SS-032 Redis-backed rate limiter

v2.0

Security

Replace in-memory counter with Redis; distributed across workers.

Depends on: SS-014 Est: 3h

SS-033 Encrypted metadata blob

v2.0

Crypto

Filename, MIME, size encrypted client-side and stored opaquely; replaces any plaintext filename storage.

Depends on: SS-006 Est: 4h

SS-034 Tampered-blob handling

v2.0

Frontend

Surface AEAD failure as a dedicated "Integrity check failed" screen — no download offered.

Depends on: SS-013 Est: 2h

SS-035 404 + Unsupported browser pages

v2.0

Frontend

Custom 404 with brand; unsupported-browser fallback with min-version table.

Depends on: SS-011 Est: 2h

SS-036 Accessibility pass (WCAG 2.2 AA)

v2.0

QA

Keyboard nav, ARIA, focus rings, screen-reader labels; Lighthouse ≥ 95 .

Depends on: SS-031 Est: 6h

SS-037 Mobile breakpoints + bottom sheet

v2.0

Frontend

Modal-as-bottom-sheet under 640px; full-bleed card under 480px.

Depends on: SS-031 Est: 4h

SS-038 k6 load testing

v2.0

QA

Scripted scenario at 50 RPS sustained; p95 budget enforced in CI.

Depends on: SS-021 Est: 4h

SS-039 Sentry + structured logging

v2.0

DevOps

Sentry on backend + frontend; PII scrubbing for URL fragments; log redaction filter.

Depends on: SS-016 Est: 3h

SS-040 Documentation + landing copy

v2.0

Docs

README, CONTRIBUTING, SECURITY.md, in-product "How it works" copy.

Depends on: SS-029 Est: 5h

16 Sprint Plan

Six 2-week sprints. Total estimated effort: **≈ 162 hours**.

SPRINT	FOCUS	TICKETS	EST. HOURS
1	Infrastructure & Crypto Foundations	SS-001...SS-007	27
2	Backend API (single-shot baseline)	SS-008, SS-009, SS-010, SS-014, SS-015	20
3	Frontend baseline & component library	SS-011, SS-012, SS-013, SS-031	30
4	v2.0 features — sender control	SS-019, SS-020, SS-022, SS-023, SS-024, SS-025	25
5	v2.0 features — reliability & UX	SS-021, SS-026, SS-027, SS-028, SS-029, SS-030, SS-033, SS-034, SS-035	40
6	Hardening, CI/CD, audit, polish	SS-016, SS-017, SS-018, SS-032, SS-036...SS-040	20

17 Risk Register

RISK	LIKELIHOOD	IMPACT	MITIGATION
User loses delete link	High	Medium	Surface delete link prominently on success page; allow re-download from same browser session via in-memory state.
User loses password	Medium	High (data unrecoverable by design)	UI copy makes this explicit upfront. Recommend strong passphrases or Link-Only mode for low-sensitivity files.
Web Crypto unavailable on user browser	Low	High	Unsupported-browser page (SS-035) with explicit minimum versions.
Spaces region outage	Low	High	Status page acknowledges; restore on region recovery; consider multi-region for v2.1.
DDoS	Medium	Medium	Cloudflare in front; Nginx rate limit; Flask-Limiter on app.
Supply-chain compromise (npm/pypi)	Low	Critical	Lockfile pinning, SRI on CDN assets (none in v2.0), pip-audit + npm audit in CI, reproducible builds.
Reusing IV bug	Low	Critical (key compromise)	Single chokepoint API for IV generation; unit-test asserts uniqueness over 10^6 samples.
Fragment leak via service worker	Low	Critical	No service worker registered in v2.0; ESLint rule blocks <code>ServiceWorkerContainer</code> .
Abuse: malware distribution	Medium	Medium-Legal	Cannot inspect content by design; reliance on takedown via reported

RISK	LIKELIHOOD	IMPACT	MITIGATION
			share-link + delete-token from victim. Documented in ToS.

18 Glossary

AEAD	Authenticated Encryption with Associated Data. Provides confidentiality + integrity in one primitive.
AES-256-GCM	Symmetric AEAD cipher chosen for ShadowShare's bulk encryption. NIST SP 800-38D.
Auth Tag	The 128-bit tag GCM appends; verifies that ciphertext has not been altered. Mismatch → decryption refused.
Blind Vault	A storage server engineered to be incapable of reading what it stores.
Blob	An opaque binary object. In our system, the ciphertext + IV concatenation persisted to Spaces.
Burn-After-Read	Mode in which the server destroys a blob on first successful download.
CSPRNG	Cryptographically Secure Pseudo-Random Number Generator. Provided by <code>crypto.getRandomValues</code> .
Delete Token	Random 256-bit secret returned to the sender once at upload; lets them revoke or read status.
IV / Nonce	Initialisation Vector. 96-bit value, unique per ciphertext, prevents AES-GCM key compromise on collision.
PBKDF2	Password-Based Key Derivation Function 2. Stretches a low-entropy password into a high-entropy key.
URL Fragment	The portion of a URL after <code>#</code> . Per RFC 3986 it is never sent to the server.
Web Crypto API	Native browser cryptographic interface. Used here as the sole crypto provider.
Zero-Knowledge	Property that the operator of a service cannot recover the data the service stores.

